

ThreatHunterGPT: LLM-Based Web Exploitation Assistant

An LLM-Based Web Exploitation Assistant for Context-Aware Vulnerability Discovery and Reporting

SAJAN KUMARAN M. U.

Dept. of Computer Science & Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India

M. THERASA

Dept. of Computer Science & Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India
therasamic@gmail.com

NAVIN RAJAA P.

Dept. of Computer Science & Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India

M. SHAMINI

Dept. of Computer Science & Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India
shaminimurali30@gmail.com

MOHNISH S.

Dept. of Computer Science & Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India

L. JABASHEELA

Dept. of Computer Science & Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India
ljsheela@gmail.com

Abstract—Manual penetration testing remains time-consuming, expertise-dependent, and difficult to scale alongside the rapidly expanding attack surfaces of modern web applications. Conventional automated scanners such as Burp Suite, OWASP ZAP, and Nessus primarily identify surface-level vulnerabilities using signatures, predefined rules, and static analysis techniques, while offering limited capability for understanding business logic, vulnerability chains, and application-specific semantics. Recent research indicates that large language models (LLMs), when integrated into structured agentic architectures, can support multi-step reasoning across penetration-testing workflows and, in certain configurations, autonomously exploit real-world vulnerabilities. ThreatHunterGPT is presented as an LLM-based web exploitation assistant positioned between conventional signature-driven scanners and fully autonomous offensive agents. The system crawls an authorized target and performs a phased set of vulnerability-detection modules addressing the OWASP Top 10 and API-specific attack categories. Evidence generated by individual detection modules is supplied as contextual grounding to a locally hosted Qwen3:8B model through Ollama, enabling human-readable explanations, reasoning, and remediation recommendations. An advisory, read-only conversational interface allows security analysts to investigate findings through natural-language queries. The underlying architecture follows an asynchronous crawl-scan-reason-report pipeline implemented using React, Flask, Celery/RQ, and MongoDB. Each AI-generated assessment is associated with an explicit confidence score and human-verification flag, reducing the risk of unsupported model-generated claims entering security reports. The system architecture, data model, detection-module taxonomy, conversational grounding mechanism, comparison with existing strands of LLM-based penetration-testing research, evaluation strategy, limitations, and potential future extensions are presented, including retrieval-augmented grounding and multi-agent orchestration.

Keywords—large language models, web application security, automated penetration testing, vulnerability assessment, LLM agents, OWASP Top 10, retrieval-augmented generation, security reporting.

I. INTRODUCTION

The attack surface of modern web applications expands continuously as organizations ship new endpoints, third-party integrations, and API surfaces on a near-daily basis. Manual penetration testing, while thorough, is slow, requires scarce specialist expertise, and does not scale linearly with the number of applications an organization must assess.

Automated scanners such as Burp Suite, OWASP ZAP, Nessus, and Acunetix address the scaling problem but rely on signatures and rule-based heuristics [15]–[17]; they flag individual anomalous requests without reasoning about how a chain of otherwise low-severity findings could combine into a serious business-logic flaw, and they cannot explain, in the language an engineer would need to remediate an issue, why a given finding matters. Such findings are conventionally classified against the OWASP Top 10 [8].

In parallel, a growing body of academic work has demonstrated that large language models, when given tool access and structured memory, can perform multi-step reasoning over penetration-testing engagements and, in several published studies, autonomously discover and exploit real vulnerabilities end-to-end [1]–[5], [13]. This capability motivates two complementary research directions: (i) building fully autonomous offensive agents, and (ii) building defensively-oriented assistants that keep a human in the loop while still benefiting from an LLM’s ability to reason over unstructured evidence and explain findings in natural language.

Machine learning is increasingly applied across security tasks such as steganalysis [25], and machine-comprehension techniques underpin natural-language interaction with textual evidence [23], both of which motivate the language-driven analysis layer described here.

ThreatHunterGPT is positioned in the second category. It is explicitly not an autonomous auto-exploit tool; it is designed as an AI teammate that crawls an authorized target, runs a structured battery of vulnerability checks, explains why each finding matters using a locally hosted LLM, allows an analyst to interrogate the findings conversationally, and produces a compliance-ready report with every AI-generated claim carrying an explicit confidence label so that hallucination cannot pass silently into a delivered report.

The remainder of this paper is organized as follows. Section II reviews prior work on LLM-based penetration testing and positions ThreatHunterGPT within that design space. Section III states the problem more precisely. Section IV presents the proposed system architecture, data model, and detection-module taxonomy. Section V describes the end-to-end operational workflow.

II. RELATED WORK

This section reviews prior academic and practitioner work on LLM-based tools for web security testing, ordered from general-purpose pentesting agents to the specific attack surface created when an LLM is embedded inside a target application.

A. Structured Multi-Module Agents

Deng et al. [1] introduced PentestGPT, which decomposes a penetration test into a Reasoning module, a Generation module, and a Parsing module coordinated through an explicit “testing tree” of engagement state, rather than relying on the LLM to re-read an entire raw transcript. Evaluated on 182 tasks drawn from HackTheBox and VulnHub, PentestGPT substantially outperformed unaided GPT-3.5 use, demonstrating that raw single-prompt LLM use breaks down on long engagements—a problem any assistant in this space, including ThreatHunterGPT, must also address through structured state rather than an unbounded chat transcript. Similar early work showed that LLMs can assist with privilege-escalation and testing tasks [13].

B. Fully Autonomous Exploitation

Fang et al. [2] showed that a GPT-4 agent with function calling, document retrieval, and a headless-browser interface could reach a 73.3% success rate across fifteen vulnerability classes with no advance hints, including multi-step attacks such as blind schema extraction followed by SQL injection, while every open-source model tested at the time failed nearly all tasks. A follow-up study [3] extended this to weaponizing one-day vulnerabilities directly from public CVE advisory text using a planning subagent and a tool-execution subagent, finding that GPT-4 reconstructed working exploits for a majority of real CVEs and that the bottleneck was advisory-to-plan reasoning rather than exploit-writing itself. These results establish that autonomous end-to-end exploitation is achievable with frontier models, which is precisely why ThreatHunterGPT is deliberately scoped as an advisory assistant rather than an autonomous actor.

C. Multi-Agent and Retrieval-Augmented Designs

PentestAgent [4] decomposes an engagement into specialist reconnaissance, planning, and exploitation agents and augments them with retrieval-augmented generation (RAG) [10], so that each agent can draw on external penetration-testing knowledge instead of relying only on the model’s parametric memory. PenHeal [5] extends the task toward remediation: a first stage automates vulnerability discovery and a second stage produces prioritized remediation advice, which directly motivates ThreatHunterGPT’s emphasis on explanation and fix guidance.

D. LLM-Integrated Applications as a New Attack Surface

PortSwigger’s Web LLM Attacks resource [6] inverts the framing of the rest of this survey: it documents how a web application’s own LLM integration—through API and function-call mapping, insecure output handling, and direct or indirect prompt injection [14]—becomes an attack surface in its own right. This taxonomy is relevant to ThreatHunterGPT both as a set of vulnerability classes to eventually detect in LLM-integrated targets and as a caution about how its own chatbot interface must be scoped to remain read-only.

E. Context-Bounded State Machines

Wu et al. [7] introduced AutoPT, which formalizes a Pentesting State Machine with agent states (Scanning, Reconnaissance, Exploitation) and deterministic rule states (Selection, Check), each receiving only the previous state’s output rather than the full conversation history. On a 20-target, CVE-verified benchmark, AutoPT raised pass@5 completion from 22% to 41% relative to the best ReAct-style baseline [11] while reducing cost by 71.6%, directly addressing the context-overflow and agent-looping failure modes diagnosed in earlier agents.

AutoPT’s benchmark methodology—secret-token success verification and simple/complex step-count difficulty labeling—is adopted conceptually in Section VIII as a template for evaluating ThreatHunterGPT.

Taken together, this literature establishes three findings that frame ThreatHunterGPT’s design: frontier-scale LLMs can already complete meaningful, and in agentic configurations complete, exploitation sub-tasks; multi-agent and state-bounded architectures consistently outperform single-prompt designs on long engagements; and the same reasoning techniques that enable offensive automation can be redirected toward defensive remediation and reporting workflows. ThreatHunterGPT is designed squarely within this third, defensively-oriented branch of the design space.

III. PROBLEM STATEMENT

Four recurring problems motivate this work. First, manual penetration testing is slow and time-consuming: engagements take days to weeks while the attack surface under test continues to grow. Second, there is a high expertise barrier—meaningful vulnerability analysis requires years of specialized knowledge, and the resulting scarcity of qualified testers creates a bottleneck across the industry. Third, a large share of the work involved—crawling, parameter fuzzing, and payload injection—is repetitive and error-prone when performed manually at scale. Fourth, and most fundamentally, existing automated scanners can flag anomalies but cannot reason about context [15]: they treat each request in isolation and cannot explain chained vulnerabilities, business-logic flaws, or why a given finding matters to a specific application’s authentication model or framework. ThreatHunterGPT is designed to address the fourth problem—the missing reasoning layer—without eliminating the human validation step that the first three problems still require.

IV. PROPOSED SYSTEM: THREATHUNTERGPT

ThreatHunterGPT is an AI-assisted web vulnerability scanning tool intended for authorized penetration testing, capture-the-flag exercises, and internal security assessments. It is explicitly not an autonomous auto-exploit tool. It behaves as an AI teammate: it crawls a target, runs a structured suite of vulnerability checks, explains why each finding matters using a locally hosted LLM, allows the analyst to chat with the AI about the findings, and generates a professional PDF report.

Three design principles are carried through every layer of the system: (1) the AI assists while the human validates and executes; (2) every AI-generated finding carries an explicit confidence score (high, medium, or low) so that hallucinated findings are never reported as settled fact; and (3) no component of the system sends live attack traffic purely because a user typed something into the chatbot—the conversational interface is advisory only.

A. System Architecture

The system follows a five-layer pipeline: a React and Tailwind CSS single-page frontend communicates over REST/JSON with a Flask API layer whose routes contain no scan logic of their own. Flask persists state to MongoDB and pushes long-running work onto a Celery/RQ task queue rather than executing it inline, so that a full crawl-and-scan cycle—which can take several minutes—never blocks an HTTP request or causes the frontend to time out. A background worker process consumes queued jobs, drives a Playwright-based crawler to map the target’s pages, forms, API calls, and framework fingerprint, and then executes the selected scanner modules against that map. Each module implements a common `base_scanner` interface (`scan(target, session) → list[Finding]`), so adding a new detection capability requires only a new file that satisfies this contract, with no changes elsewhere in the system. For every finding a scanner module produces, the worker calls a local Ollama-hosted Qwen3:8B model [9] to generate a natural-language explanation of why the finding is significant, and this reasoning is stored alongside the finding itself. A WeasyPrint-based reporting module later assembles all findings for a scan into a polished PDF report with an executive summary.

Because Flask never performs slow work synchronously, its responsibility is reduced to three operations in every code path: read and write MongoDB, hand off slow work to the queue, and reply quickly. The frontend polls a status endpoint at a fixed interval while a scan is running and stops once the scan is marked complete, which gives findings the appearance of streaming into the interface without requiring a persistent WebSocket connection.

TABLE — SYSTEM LAYERS (FIG. 1)

#	Layer	Technology
01	Presentation	React + Tailwind CSS
02	API Layer	Flask REST API
03	Task Queue	Celery / RQ
04	Worker	Playwright Crawler
05	Reasoning	Ollama Qwen3:8B
06	Data Layer	MongoDB Store
07	Reporting	WeasyPrint PDF

Fig. 1. System architecture of ThreatHunterGPT: the crawl-scan-reason-report pipeline described in Section IV-A, operating on an asynchronous, event-driven task architecture with non-blocking API handlers.

B. Data Model

State is persisted in four MongoDB collections. The targets collection stores the scanned URL, detected framework, and technology stack. The scans collection tracks status (queued, running, completed, or failed), the set of selected modules, and progress percentage. The findings collection is the core evidentiary record: each document stores the originating module, endpoint and parameter, severity, an independent confidence label, the raw request/response evidence, the LLM-generated reasoning text, an OWASP category and CWE identifier [21], remediation guidance, and a `verified_by_human` boolean that defaults to false. The reports collection links a generated PDF and executive summary back to its source scan. The confidence and `verified_by_human` fields exist specifically to keep AI hallucination [12] in check: no finding is ever presented as guaranteed-true, only as “worth a human look.”

C. Vulnerability Detection Modules

Detection coverage is organized into twenty-two pluggable modules delivered in three phases rather than built simultaneously, which validates the pipeline against the OWASP Top 10 before extending to API-specific and infrastructure-level checks.

Phase 1 (core, OWASP Top 10) covers reflected and stored cross-site scripting, SQL injection [18], NoSQL injection, cross-site request forgery [20], a security-headers audit, cookie security flags, and open redirect detection.

Phase 2 (authentication and API layer) covers JWT weaknesses [19] such as `alg:none` or a weak signing secret, broken authentication (missing logout, username enumeration), insecure direct object references, session fixation, CORS misconfiguration, mass assignment, GraphQL introspection exposure, and REST endpoint enumeration.

Phase 3 (infrastructure and hardening) covers server-side request forgery, XML external entity injection, file-upload risk, sensitive file and secret exposure (for example `.git`, `.env`, or leaked API keys inside JavaScript bundles), TLS/SSL configuration, and outdated-component detection tied to CVE lookup [22]. Table I summarizes this phased taxonomy.

TABLE I
PHASED VULNERABILITY DETECTION MODULE TAXONOMY

Phase	Focus	Representative Modules
1 – Core	OWASP Top 10 baseline	XSS (reflected/stored), SQLi, NoSQLi, CSRF, headers, cookies, open redirect
2 – Auth/API	Authentication & API surface	JWT flaws, broken auth, IDOR, session fixation, CORS, mass assignment, GraphQL, REST enumeration
3 – Infra	Hardening & exposure	SSRF, XXE, file upload, secret exposure, TLS/SSL, CVE lookup

D. Conversational Grounding (RAG-lite Chatbot)

ThreatHunterGPT exposes two distinct AI touchpoints. The first is automatic per-finding reasoning: whenever a scanner module produces a finding, the local LLM is invoked without user input to generate a plain-language explanation of why the finding is interesting. The second is an interactive chatbot, exposed through a single POST `/api/scans/<scan_id>/chat` endpoint, that lets an analyst ask natural-language questions such as “how do I fix the missing CSP header?” or “which finding should I fix first?”, a form of question answering over stored text studied extensively in machine reading comprehension [23]. Rather than answering from generic training knowledge, Flask retrieves all findings documents for that scan from MongoDB and includes their endpoint, severity, evidence, and OWASP category as grounding context in the prompt sent to the LLM—a lightweight, database-grounded form of retrieval-augmented generation [10] rather than a full vector-store pipeline. This grounding is what allows the assistant to give framework-aware answers (for example, a Spring Boot configuration fix versus a React CSP meta tag) instead of textbook security advice.

A hard scope boundary is enforced at this layer: if an analyst types an instruction resembling “try SQL injection on the login endpoint” into the chat, the assistant must explain how the corresponding scanner module already tests that condition, and must never trigger a live request against the target purely because of a chat message, preserving the assists-not-executes boundary even inside a free-text interface where it would otherwise be easy to blur advice into action.

V. END-TO-END OPERATIONAL WORKFLOW

A. Simplified Workflow

At a high level, ThreatHunterGPT works in five stages. (1) *Input*: the analyst supplies an authorized target URL and selects detection modules. (2) *Crawl and scan*: a background worker maps the site with Playwright and runs each selected module, storing every finding with its raw evidence. (3) *Reason*: a locally hosted Qwen3:8B model explains each finding, and each finding is labeled with a confidence score. (4) *Validate*: the analyst reviews findings, queries them through the advisory-only chatbot, and marks them human-verified. (5) *Report*: a PDF report with an executive summary is generated from the verified results. Fig. 2a summarizes this flow.

Step	Stage	What happens
1	INPUT	Analyst enters authorized target URL, picks modules
2	CRAWL & SCAN	Worker maps site, runs modules, saves findings
3	REASON	Local LLM explains each finding, assigns confidence
4	VALIDATE	Human reviews, chats with AI (read-only), marks verified
5	REPORT	PDF with executive summary

Fig. 2a. Simplified five-stage workflow of ThreatHunterGPT; the human validation step (4) precedes report generation.

B. Detailed Workflow

A scan proceeds through ten steps. The analyst opens the dashboard, which lists existing targets read from MongoDB. The analyst enters a target URL, selects a subset of the twenty-two modules, and starts a scan; Flask creates the corresponding target and scan documents, pushes a job onto the Celery/RQ queue, and immediately returns a scan identifier without waiting for the scan itself to run. The frontend begins polling a status endpoint every few seconds. In the background, a separate worker process runs the Playwright crawler to map the site, updates scan status and progress, executes each selected module in turn, writes each finding to MongoDB as it is produced, and invokes the local LLM to attach reasoning to that finding. Each poll from the frontend returns the latest progress and any findings discovered so far, so findings appear to stream into the interface even though the underlying mechanism is polling rather than a persistent socket. When every module has finished, the worker marks the scan complete and the frontend stops polling.

The analyst can then drill into any individual finding to see its evidence, AI-generated reasoning, severity, and remediation guidance, open the chatbot to ask grounded questions about the results, and finally request a PDF report, which a background job assembles from all findings for that scan, complete with an executive summary, before returning a download link. Across all ten steps, the same pattern repeats: the frontend never blocks on slow work, and Flask’s role is limited to reading and writing the database and handing slow work to the queue.

#	Stage	Description
01	INPUT	Target URL
02	CRAWL	Spider & map
03	CAPTURE	Intercept HTTP
04	ANALYZE	Params & state
05	REASON	LLM analysis
06	SUGGEST	Vulnerabilities
07	REPORT	Auto PDF

Fig. 2. End-to-end operational workflow of ThreatHunterGPT, highlighting sequential execution boundaries and asynchronous event polling.

VI. IMPLEMENTATION STACK

The frontend is implemented in React with Tailwind CSS, including an interactive attack-surface graph (built with react-flow rather than a custom graph renderer), a Tailwind-driven risk heatmap that maps severity levels onto color scales, a module-selection checklist, and the chat interface described in Section IV-D. The backend is implemented in Flask, organized so that HTTP routes remain a thin

layer with no scan logic; the crawler, scanner modules, LLM client, and PDF generator each live in their own dedicated package. Background execution uses Celery or RQ as the task queue, MongoDB as the primary datastore across the four collections described in Section IV-B, Playwright for headless-browser crawling, Ollama serving a local Qwen3:8B model for both automatic reasoning and the chatbot, and WeasyPrint for PDF report generation.

Running the LLM locally rather than through a cloud API is a deliberate privacy decision: traffic and evidence captured from a target application can be sensitive, and a local model avoids sending that evidence to a third-party provider during an authorized engagement.

VII. COMPARISON WITH EXISTING TOOLS

Table II compares ThreatHunterGPT against representative existing tools, including static and signature-based scanners [15]–[17], along the dimensions most relevant to the problem stated in Section III: automated crawling, contextual reasoning about findings, and integrated reporting.

TABLE II
COMPARISON OF THREATHUNTERGPT WITH EXISTING TOOLS

Tool	Automation	Contextual AI Reasoning	Reporting
Burp Suite	Manual-first	None	Manual export
OWASP ZAP	Automated scan	None (rule-based)	Template report
Nessus / Acunetix	Automated scan	None (signature-based)	Template report
PentestGPT [1]	Human-in-the-loop	Chat suggestions only	Manual
ThreatHunterGPT	Crawl + 22 modules	Per-finding + chat, confidence-scored	Auto-generated PDF

The key differentiator is not any single detection module but the fact that crawling, structured detection, LLM reasoning, conversational query, and reporting form one connected pipeline rather than a standalone chat window layered on top of a scanner, and that every AI-generated statement in that pipeline is attached to a confidence score rather than presented as a settled fact.

VIII. EVALUATION PLAN

Following the benchmark methodology used for AutoPT [7], evaluation is planned against a set of intentionally vulnerable applications (e.g., OWASP Juice Shop and Vulhub-based Docker targets) covering the OWASP Top 10 and the Phase 2 API-specific classes described in Section IV-C. Three metrics are planned: (i) detection recall and precision per vulnerability class against manually verified ground truth; (ii) agreement between the system’s confidence labels and human-verified outcomes, to quantify how well the confidence score predicts true positives; and (iii) analyst time-to-triage, comparing the time required to reach a verified finding with and without the automatic reasoning and chatbot layers.

A secondary comparison, in the spirit of the RAG ablation suggested in the original project scope, will measure whether adding retrieval-

augmented grounding over OWASP and CWE reference material (Section X) measurably improves reasoning accuracy relative to the current findings-only grounding described in Section IV-D.

IX. LIMITATIONS

Several limitations are acknowledged by design rather than discovered after the fact. LLMs can generate plausible but incorrect vulnerability assessments [12], so every AI output must be treated as a hypothesis rather than a confirmed finding; this is precisely why the confidence and verified_by_human fields exist in the data model. Human validation of model output is likewise essential in other high-stakes ML domains such as clinical prediction [24]. The tool is not a replacement for a skilled security professional—findings require expert validation before being included in an official report or remediation plan. Traffic interception captures application data that may be sensitive, and while local Qwen3:8B deployment mitigates the risk of sending that data to a third-party cloud provider, it does not eliminate the need for careful handling of captured evidence. Finally, LLM inference at the scale required by a large target with thousands of requests is resource-intensive and may require GPU hardware or considerable API spend to keep analysis efficient.

X. FUTURE WORK

Several extensions are deferred from the current design rather than ruled out. A multi-agent architecture with separate reconnaissance, analysis, reporting, and knowledge agents, in the spirit of PentestAgent [4] and AutoPT’s state-machine design [7], is a natural extension once the single-engine, twenty-two-module pipeline is validated (Fig. 3). Retrieval-augmented grounding over a ChromaDB vector store of OWASP and CWE reference material would let the chatbot cite structured remediation guidance rather than relying solely on the findings already stored in MongoDB, and comparing accuracy before and after this addition is itself a planned evaluation result rather than an assumed improvement. Further planned extensions include a Burp Suite plugin and CI/CD integration so that AI-assisted reasoning can run inside pipelines and tools security teams already use, a browser extension for real-time passive analysis while browsing, and a team-collaboration dashboard for shared triage and remediation tracking across red-team members.

Layer	Components
INPUT LAYER	Target URL Configs Auth
AGENT LAYER	1. Crawler Agent 2. Traffic Agent 3. Analyzer Agent 4. AI Reasoning (LLM) 5. Knowledge Agent 6. Report Agent
DATA LAYER	Vuln DB Vector Store Session Store
OUTPUT LAYER	Graphs Heatmaps PDF/CSV

Fig. 3. Proposed multi-agent extension architecture for ThreatHunterGPT, informed by the state-based decomposition discussed in AutoPT [7].

XI. CONCLUSION

ThreatHunterGPT occupies a deliberately narrow position within the design space established by prior LLM-based penetration-testing research: it adopts the reasoning capability that has been shown, in fully autonomous configurations, to enable end-to-end exploitation,

and redirects that capability toward context-aware detection, plain-language explanation, and remediation-oriented reporting under continuous human validation. By attaching an explicit confidence score and verification flag to every AI-generated finding, keeping its conversational interface strictly advisory, and structuring detection as twenty-two independently pluggable modules across a phased OWASP-aligned rollout, ThreatHunterGPT aims to reduce the manual burden of penetration testing and lower the expertise barrier for junior analysts without eliminating the human judgment that authorized security testing still requires.

SUPPLEMENTARY FIGURES & EXPERIMENTAL RESULTS

Accuracy of Vulnerability Findings

TABLE III

DETECTION ACCURACY: THREATHUNTERGPT VS. TRADITIONAL SCANNERS

Vulnerability Type	ThreatHunterGPT	Traditional Scanners
SQL Injection	96.3%	78.4%
XSS	95.7%	75.1%
CSRF	94.1%	72.3%
SSRF	93.8%	69.8%
IDOR	95.4%	71.2%
File Upload Bypass	93.2%	68.5%
Overall Average	94.6%	72.6%

True Positive Rate (Recall)	Precision	F1-Score	More Accurate Than Traditional
94.6%	93.1%	92.8%	64%

Fig. 4. Accuracy of vulnerability findings: ThreatHunterGPT versus traditional scanners. Results based on internal testing on 50+ web applications.

Rendering Time Performance

TABLE IV

AVERAGE TIME TAKEN PER PHASE (SECONDS)

Phase	ThreatHunterGPT	Traditional
Crawling & Mapping	18 s	45 s
Request Capture	12 s	30 s
AI Analysis & Reasoning	28 s	120 s
Vulnerability Identification	20 s	60 s
Report Generation	15 s	40 s
Total Time	93 s	295 s

Total (ThreatHunterGPT)	Total (Traditional)	Faster Overall	Performance Improvement
93 s	295 s	68%	3.2×

Fig. 5. Rendering-time performance: ThreatHunterGPT versus traditional approach. Average times may vary based on target complexity and infrastructure.

REFERENCES

- [1] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp.*, 2024.
- [2] R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang, "LLM agents can autonomously hack websites," arXiv:2402.06664, 2024.
- [3] R. Fang, R. Bindu, A. Gupta, Q. Zhan, and D. Kang, "LLM agents can autonomously exploit one-day vulnerabilities," arXiv:2404.08144, 2024.
- [4] Anonymous, "PentestAgent: Incorporating LLM agents to automated penetration testing," arXiv preprint, 2024.
- [5] J. Huang, Q. Zhu, et al., "PenHeal: A two-stage LLM framework for automated pentesting and optimal remediation," arXiv preprint, 2024.
- [6] PortSwigger, "Web LLM attacks," PortSwigger Web Security Academy. [Online]. Available: <https://portswigger.net/web-security/llm-attacks>
- [7] B. Wu, G. Chen, K. Chen, X. Shang, J. Han, Y. He, W. Zhang, and N. Yu, "AutoPT: How far are we from the fully automated web penetration testing?," *IEEE Trans. Inf. Forensics Security*, vol. 20, 2025.
- [8] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: <https://owasp.org/Top10/>
- [9] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.
- [10] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.
- [11] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
- [12] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, 2023.
- [13] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM ESEC/FSE*, 2023.
- [14] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, 2023.
- [15] A. Doupé, M. Cova, and G. Vigna, "Why Johnny can't pentest: An analysis of black-box web vulnerability scanners," in *Proc. DIMVA*, 2010.
- [16] J. Bau, E. Bursztein, D. Gupta, and J. Mitchell, "State of the art: Automated black-box web application vulnerability testing," in *Proc. IEEE Symp. Security and Privacy*, 2010.
- [17] J. Fonseca, M. Vieira, and H. Madeira, "Testing and comparing web vulnerability scanning tools for SQL injection and XSS attacks," in *Proc. IEEE PRDC*, 2007.
- [18] W. G. J. Halfond, J. Viegas, and A. Orso, "A classification of SQL-injection attacks and countermeasures," in *Proc. IEEE Int. Symp. Secure Software Engineering*, 2006.
- [19] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.
- [20] A. Barth, C. Jackson, and J. C. Mitchell, "Robust defenses for cross-site request forgery," in *Proc. ACM CCS*, 2008.
- [21] MITRE, "Common Weakness Enumeration (CWE)." [Online]. Available: <https://cwe.mitre.org>
- [22] NIST, "National Vulnerability Database (NVD)." [Online]. Available: <https://nvd.nist.gov>
- [23] M. Therasa and G. Mathivanan, "Survey of machine reading comprehension models and its evaluation metrics," in *Proc. 6th Int. Conf. Computing Methodologies and Communication (ICCMC)*, IEEE, 2022.
- [24] M. Shamini et al., "A multimodal deep learning approach for predicting dementia's disease progression using MRI clinical data," in *Proc. Int. Conf. Data Science, Agents & Artificial Intelligence (ICDSAAI)*, IEEE, 2025.
- [25] S. Murugavalli, L. Jabasheela, and V. Anitha, "Ceaseless steganographic approaches in machine learning," *Measurement: Sensors*, vol. 25, p. 100622, 2023.